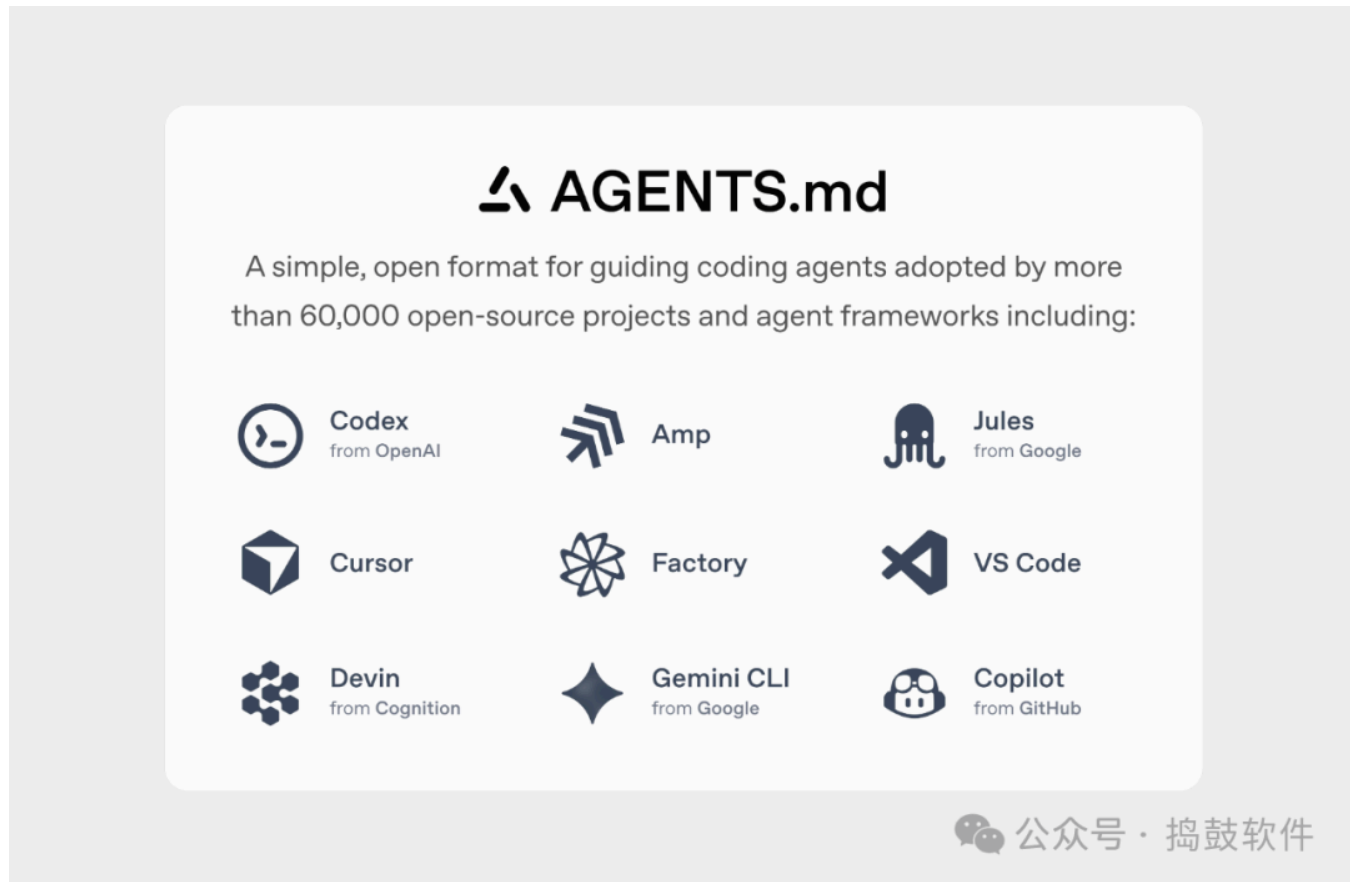


11.1k stars ! AGENTS.md : 让AI编程助手真正懂你的项目 !

mp.weixin.qq.com/s/CrR7FKH5ePdQW5Bb2fptBg



AGENTS.md : 让AI编程助手真正懂你的项目

前言

当你用 GitHub Copilot、Cursor 或 Claude 等 AI 编程助手写代码时，是否遇到过这样的困扰：AI 不知道你项目的构建命令、不清楚代码规范、频繁修改不该碰的文件？

如果把 AI 编程助手比作新来的实习生，那么 AGENTS.md 就是你给他准备的"入职指南"——让 AI 真正理解你的项目，按照你的方式工作。

一、解决了什么问题？

1. AI 编程助手的"失明"困境

当前的 AI 编程工具虽然强大，但它们对你的项目一无所知：

- 不知道如何构建：运行什么命令？用 npm 还是 pnpm？
- 不懂代码规范：用 Tab 还是空格？单引号还是双引号？

- • **缺乏上下文**：项目结构是什么？哪些文件不能动？
- • **测试方式不明**：如何运行测试？CI/CD 流程是什么？

这导致 AI 助手经常“好心办坏事”：修改了不该改的配置文件、使用了错误的命令、违反了团队的编码规范。

2. 配置文件的混乱现状

更糟糕的是，每个 AI 工具都要求不同的配置文件：

- • .cursorrules - Cursor 专用
- • .windsurfrules - Windsurf 专用
- • .claude rules - Claude 专用
- • .copilot-instructions.md - Copilot 专用

开发者需要维护多个文件，每当新工具出现，就要新增一份配置。这种碎片化浪费时间，也增加了维护成本。

3. AGENTS.md 的解决方案

AGENTS.md 提供了一个简单、开放的格式来指导 AI 编程助手，就像给 AI 准备的 README 文件：

- ✓ 一份文件，通用所有工具：GitHub Copilot、Cursor、Claude、Windsurf 等主流工具都支持
- ✓ 统一的项目知识库：所有 AI 助手读取同一份指南
- ✓ 简单的 Markdown 格式：无需学习复杂语法，人人可写
- ✓ 逐步完善：从简单开始，根据实际需求迭代添加

二、AGENTS.md 是什么？

核心理念

AGENTS.md 是为 AI 编程助手准备的 README——一个专门的、可预测的位置，提供上下文和指令帮助 AI 更好地工作。

与 README.md 的区别：

README.md	AGENTS.md
面向人类开发者	面向 AI 编程助手
项目介绍、快速开始	详细的构建步骤、测试流程
贡献指南	代码规范、工作流程
简洁优先	详尽的技术细节

标准格式

AGENTS.md 使用标准 Markdown 格式，通常包含以下部分：

项目名称

构建命令部分：

- • 安装依赖：npm install
- • 启动开发服务器：npm run dev
- • 运行测试：npm test
- • 生产构建：npm run build

代码规范部分：

- • TypeScript 严格模式
- • 使用单引号，不加分号
- • 优先使用函数式编程模式
- • 2 空格缩进

测试说明部分：

- • 运行所有测试：npm test
- • 运行单个测试：npm test -- 文件名
- • 提交前必须通过所有测试

项目结构部分：

- • /src - 源代码
- • /tests - 测试文件
- • /docs - 文档

禁止操作部分：

- • 不要修改 package-lock.json
- • 不要删除失败的测试
- • 不要修改 .github/workflows 目录

支持的工具

目前已支持 AGENTS.md 的主流工具包括：

- • **GitHub Copilot** - 微软出品的 AI 编程助手
- • **Cursor** - 新一代 AI 代码编辑器
- • **Windsurf** - Codeium 推出的 AI 编程工具
- • **Claude** (via Cline) - Anthropic 的 AI 助手
- • **Aider** - 命令行 AI 编程工具
- • **Devin** - Cognition 的自主 AI 工程师

AGENTs.md

A simple, open format for guiding coding agents adopted by more than 60,000 open-source projects and agent frameworks including:



Codex
from OpenAI



Amp



Jules
from Google



Cursor



Factory



VS Code



Devin
from Cognition



Gemini CLI
from Google



Copilot
from GitHub

 公众号 · 捣鼓软件

main3 Branches0 Tags

Go to file

Code

romainhuet Add note on AAIF in the About section ✓40de070 · 2 days ago 29 Commits

components	Add note on AAIF in the About section	2 days ago
pages	Update favicon to new AGENTS.md logo	3 days ago
public	Update social card to new AGENTS.md logo	3 days ago
styles	feat: better handle the growing list of editors (#85)	2 months ago
.gitignore	Initial commit	4 months ago
AGENTS.md	Initial commit	4 months ago
LICENSE	Initial commit	4 months ago
README.md	Update README to new AGENTS.md logo	3 days ago
next-env.d.ts	Initial commit	4 months ago
next.config.ts	Initial commit	4 months ago
package.json	chore: bump next.js for cve-2025-55182 (#112)	5 days ago
pnpm-lock.yaml	chore: bump next.js for cve-2025-55182 (#112)	5 days ago
postcss.config.mjs	Initial commit	4 months ago
tsconfig.json	Initial commit	4 months ago

READMEMIT license

AGENTS.md



A simple, open format for guiding coding agents adopted by more than 60,000 open-source projects and agent frameworks including:

About

AGENTS.md — a simple, open format for guiding coding agents

[agents.md](#)

Readme

MIT license

Activity

Custom properties

11.1k stars

88 watching

823 forks

Report repository

Releases

No releases published

Packages

No packages published

Contributors 19

+ 5 contributors

Languages

TypeScript 94.3%

CSS 5.5%

JavaScript 0.2%

公众号 · 捣鼓软件

三、怎么用？

1. 创建 AGENTS.md 文件

在项目根目录创建 AGENTS.md 文件：

```
touch AGENTS.md
```

2. 编写基础内容

从最重要的信息开始，成功的 AGENTS.md 会把可执行命令放在前面的部分，包含 npm test、npm run build 等，要包含具体的标志和选项，而不只是工具名称。

一个基础的 AGENTS.md 示例：

```
# 我的项目 ## 开发命令
- 安装依赖 : pnpm install
- 启动开发 : pnpm dev (运行在 http://localhost:3000)
- 运行测试 : pnpm test --run
- 类型检查 : pnpm typecheck
- 代码检查 : pnpm lint ## 代码风格
- 使用 TypeScript 严格模式
- Prettier 自动格式化
- ESLint 规则见 .eslintrc.js ## 重要约定
- 组件放在 /src/components
- 工具函数放在 /src/utlis
- 测试文件命名 : *.test.ts
```

3. 添加具体示例

一个真实的代码片段展示你的风格，胜过三段文字描述。添加代码示例帮助 AI 理解你的编码风格。

比如在代码风格示例章节，可以展示 React 组件的推荐写法和应该避免的写法，使用 TypeScript 定义清晰的 Props 类型，采用函数式组件等。

4. 设置明确边界

告诉 AI 什么不该做：

```
## 禁止操作
- ⓧ 不要修改 package-lock.json 或 pnpm-lock.yaml
- ⓧ 不要删除或跳过失败的测试
- ⓧ 不要修改 /config 目录下的配置文件
- ⓧ 只在 /tests 目录写测试，不要修改源代码
```

5. 使用 AI 工具

文件创建后，AI 编程助手会自动读取。

在 **GitHub Copilot** 中：

使用 `@workspace` 命令并引用 `AGENTS.md`，例如“根据 `AGENTS.md` 添加一个新功能”

在 **Cursor** 中：

直接在提示中提到，例如“按照 `AGENTS.md` 的规范重构这个组件”

在命令行 (**Aider**) 中：

```
aider --read AGENTS.md
```

6. 高级技巧：嵌套 `AGENTS.md`

对于 Monorepo 项目，可以在子项目中创建特定的 `AGENTS.md`：

```
my-monorepo/
├── AGENTS.md          # 全局配置
├── packages/
│   ├── frontend/
│   │   └── AGENTS.md  # 前端特定配置
│   └── backend/
│       └── AGENTS.md  # 后端特定配置
```

子目录的 AGENTS.md 会继承并覆盖根目录的配置。

7. 用 Copilot 生成 AGENTS.md

如果不知道怎么写，可以让 Copilot 帮你生成。给出明确的要求，比如：

创建一个测试专用的 agent，要求角色定位为 QA 工程师、只写测试代码、运行测试并分析结果、只写入 /tests/ 目录、从不修改源代码或删除失败的测试、包含良好测试结构的具体示例等。

四、最佳实践

1. 从小开始，逐步完善

不要一开始就写几千字的文档。从最核心的命令开始：

第一版：基础命令

列出安装、开发、测试等基本命令

第二版：添加规范

补充代码规范，如 TypeScript 严格模式、使用函数式组件、Props 必须定义类型等

第三版：补充边界

明确禁止操作，如不修改配置文件、不删除测试等

2. 根据 AI 的错误调整

最好的 AGENTS.md 文件是通过迭代成长的，而不是预先规划的。当 AI 犯错时，在 AGENTS.md 中明确说明。

比如记录常见错误：AI 曾错误使用 npm 而不是 pnpm，那么就明确说明本项目统一使用 pnpm 管理依赖；AI 曾修改了数据库迁移文件，那么就说明迁移文件一旦创建不可修改，只能新建。

3. 提供具体的上下文

模糊的指令不起作用，具体的指令才有效。

✗ 模糊的指令：写好的代码

✓ 具体的指令：

- • 所有 API 调用使用 axios 而非 fetch
- • 错误处理必须包含用户友好的提示信息
- • 表单验证使用 react-hook-form + zod

4. 六大核心领域

创建 AGENTS.md 时应覆盖六个核心领域：

1. **命令** - 构建、测试、开发服务器等
2. **测试** - 如何运行、如何写测试、测试覆盖率要求
3. **项目结构** - 目录组织、文件命名规则
4. **代码风格** - 格式化、命名、模式
5. **Git 工作流** - 分支策略、提交信息格式、PR 流程
6. **边界** - 什么不能做、什么文件不能碰

五、真实案例

案例 1：React + TypeScript 项目

一个完整的前端项目 AGENTS.md 示例：

项目名称： MyApp

开发环境：

- • Node.js 18+
- • pnpm 8+
- • 开发服务器：http://localhost:5173

命令列表：

- • pnpm install - 安装依赖
- • pnpm dev - 启动 Vite 开发服务器
- • pnpm build - 生产构建
- • pnpm test - 运行 Vitest 测试
- • pnpm lint - ESLint 检查
- • pnpm typecheck - TypeScript 类型检查

技术栈：

- • React 18 + TypeScript
- • Vite 5
- • TailwindCSS 3
- • React Router 6
- • Zustand (状态管理)
- • React Query (数据获取)

代码规范：

- • 函数式组件 + Hooks
- • Props 使用 TypeScript interface
- • 组件文件：PascalCase.tsx
- • 工具函数：camelCase.ts
- • 使用 Tailwind 类名，避免内联样式

项目结构：

- • src/components/ - 可复用组件
- • src/pages/ - 页面组件
- • src/hooks/ - 自定义 Hooks
- • src/utils/ - 工具函数
- • src/stores/ - Zustand stores
- • src/api/ - API 调用
- • src/types/ - TypeScript 类型

测试要求：

- • 组件测试覆盖率 > 80%
- • 测试文件：ComponentName.test.tsx
- • 使用 React Testing Library
- • 提交前必须通过所有测试

禁止操作：

- •  不使用 class 组件
- •  不直接修改 state (使用 Zustand actions)
- •  不在组件内直接调用 API (使用 React Query)
- •  不修改 vite.config.ts 除非必要

案例 2：Python 后端项目

一个完整的后端项目 AGENTS.md 示例：

项目名称：API Backend

环境设置：

- • Python 3.11+
- • Poetry 用于依赖管理

命令列表：

- • poetry install - 安装依赖
- • poetry run dev - 启动开发服务器 (http://localhost:8000)
- • poetry run test - 运行 pytest
- • poetry run lint - Ruff 检查
- • poetry run format - Black 格式化

技术栈：

- • FastAPI 0.104+
- • SQLAlchemy 2.0 (ORM)
- • Alembic (数据库迁移)
- • Pydantic V2 (数据验证)
- • PostgreSQL 15

代码规范：

- • 类型注解必须完整
- • 使用 Pydantic models 验证输入
- • 数据库操作使用 async/await
- • 遵循 PEP 8 (由 Ruff + Black 保证)

项目结构：

- • app/api/ - API 路由
- • app/models/ - SQLAlchemy models
- • app/schemas/ - Pydantic schemas
- • app/services/ - 业务逻辑
- • app/db/ - 数据库配置
- • app/tests/ - 测试

数据库迁移：

- • 创建迁移：poetry run alembic revision --autogenerate -m "描述"
- • 应用迁移：poetry run alembic upgrade head
- • 回滚：poetry run alembic downgrade -1

测试要求：

- • 覆盖率 > 85%
- • 使用 pytest fixtures
- • API 测试使用 TestClient
- • 数据库测试使用独立的测试数据库

禁止操作：

- •  不修改已应用的迁移文件
- •  不在 API 路由中直接写 SQL
- •  不在 models 中写业务逻辑
- •  不跳过类型注解

六、总结

AGENTS.md 正在成为 AI 辅助编程时代的新标准，它解决了一个简单但关键的问题：**如何让 AI 编程助手真正理解你的项目。**

核心价值

- ✓ **统一标准**：一份文件，所有 AI 工具通用
- ✓ **提升效率**：AI 少犯错，开发更顺畅
- ✓ **降低成本**：不用维护多份配置文件
- ✓ **易于上手**：简单的 Markdown，5 分钟就能写完第一版

立即行动

1. 在你的项目根目录创建 AGENTS.md
2. 从基础命令和代码规范开始
3. 根据实际使用情况逐步完善
4. 观察 AI 助手的表现提升

未来展望

随着越来越多的开发工具和团队采用 AGENTS.md，它正在成为开源项目的标配文件。GitHub、Anthropic、Codeium 等公司都已支持这一标准。

在 AI 编程助手成为标配的今天，AGENTS.md 就像项目的"说明书"——让每一个 AI 助手都能快速上手，按照你的方式工作。

相关链接：

- 官方网站：<https://agents.md/>
- GitHub 仓库：<https://github.com/agentsmd/agents.md>
- 更多示例：<https://agents.md/> 查看社区分享